

REFACTORTACTICS

Pipeline contenuti, Data Assets e validazione

ID stabili, Asset Manager, tag, hash e percorso modding-ready

Documento: PDR-09

Versione: 0.1 - Consolidato iniziale

Data: 3 agosto 2026

Baseline tecnica: Unreal Engine 5.8

PDR della supply chain dei contenuti competitivi.

Controllo del documento

Campo	Valore
Stato	Bozza consolidata per revisione tecnica
Versione	0.1 - Consolidato iniziale
Data	3 agosto 2026
Autore	RefactorTactics / documentazione consolidata con supporto AI
Regola di prevalenza	Decisioni esplicite del progetto > requisiti consolidati > proposte PDR > ricerca web di supporto

Sommario esecutivo

I contenuti sono Primary Data Assets catalogati con ID e versioni stabili. Prima di un match vengono scoperti, validati, risolti in un manifest e hashati. Il modding pubblico resta fuori scope, ma le fondamenta evitano hard-code e riferimenti fragili.

Decisioni consolidate

Gameplay Tags governati; validator per ID, dipendenze, mappe, limiti e cooking; nessun codice nativo non fidato nelle mod retail.

Assunzioni operative

Lo schema JSON mod e il formato esatto del manifest vengono introdotti dopo la stabilizzazione dei Data Assets interni.

Indice del documento

1. Obiettivi data-driven
2. ID e versioni
3. Primary Data Assets
4. Cataloghi
5. Gameplay Tags
6. Manifest e hash
7. Validator
8. Serialization/JSON
9. Modding-ready
10. Git LFS e repository
11. Test

1. Obiettivi data-driven

C++ definisce invarianti e possibilità; asset e Blueprint scelgono la variante. Ogni contenuto competitivo deve essere identificabile, versionato, validabile e incluso in un manifest del match.

2. ID e versioni

Concetto	Regola
Stable ID	Stringa/Name governata, mai derivata solo dal display name localizzato.
Definition Version	Incremento quando cambia semantica o serializzazione.
RulesVersion	Versione del resolver e ordine fasi.
ContentManifestHash	Hash delle definizioni effettivamente caricate per la partita.
Redirect	Mappatura esplicita per rename; vietato silenziosamente in ranked.

3. Primary Data Assets

UPrimaryDataAsset fornisce un PrimaryAssetId e supporto Asset Bundle, permettendo al progetto di scoprire e caricare definizioni tramite Asset Manager. Le classi native proposte includono CharacterDefinition, AbilityDefinition, RulesetDefinition, MapDefinition e ObjectiveDefinition.

4. Cataloghi

URContentCatalog

```
Characters: Map<CharacterId, SoftObjectPtr<CharacterDefinition>>
Abilities:  Map<AbilityId, SoftObjectPtr<AbilityDefinition>>
Rulesets:   Map<RulesetId, SoftObjectPtr<RulesetDefinition>>
Maps:       Map<MapId, SoftObjectPtr<MapDefinition>>
```

Build/Match startup:

Discover -> Validate -> Resolve dependencies -> Build manifest -> Hash -> Lock

5. Gameplay Tags

Gameplay Tags sono label gerarchiche definite in un dizionario governato. RefactorTactics separa sorgenti Config/Tags per dominio e limita la creazione libera di tag in asset.

Root	Esempi
Ability	Ability.Attack.Line, Ability.Defense.Counter
Target	Target.Cell, Target.Unit, Target.Direction
Damage	Damage.Kinetic, Damage.Electric, Damage.Fire
Status	Status.Wet, Status.Burning, Status.Guarded
Surface	Surface.Water, Surface.Metal, Surface.HighGround
Event	Event.Move.Blocked, Event.Ability.Interrupted
Phase	Phase.Movement, Phase.Attack, Phase.Environment

6. Manifest e hash

- Il server risolve tutte le definizioni richieste prima del match.
- Il manifest include ID, versioni, dipendenze e hash normalizzati.
- I client confrontano il manifest per compatibilità; il server resta fonte di verità.
- La playlist ranked blocca ruleset e contenuti ammessi.
- Il TurnLog registra almeno RulesVersion e ContentManifestHash.

7. Validator

Regola	Severità
ID duplicato o vuoto	Error
Tag ignoto/restricted non autorizzato	Error
Dipendenza mancante/ciclo non consentito	Error
Valori fuori limite, range/AoE negativi	Error
Ability senza moving-target policy	Error
Map edge verso cella inesistente	Error
Cover direction incoerente o layer invalido	Error
Soft reference non cookata nel bundle	Error
Display metadata mancante	Warning
Hash non riproducibile tra due scansioni	Error

8. Serialization e JSON

Il formato runtime Unreal può usare USTRUCT/archives; per future mod data-only si prevede un JSON versionato con schema esplicito. Non esporre subito JSON come fonte primaria del vertical slice: prima stabilizzare le definizioni native e i validator, poi aggiungere import/export deterministico.

9. Modding-ready

- Fin dal giorno uno: ID stabili, riferimenti indiretti, versioni, hash e validator.
- Fino alla beta: nessun modding pubblico; contenuti interni usano la stessa disciplina.
- Prima fase pubblica: data-only/asset-only, sandbox e whitelist di tipi/tag.
- No codice nativo non fidato nelle mod retail.
- Manifest e compatibilità di rete impediscono match tra contenuti divergenti.

10. Git LFS e repository

```
/Source          Git normale
/Config          Git normale
/Content/*.uasset Git LFS
/Content/*.umap   Git LFS
/Docs/PDR/*.md    Git normale (sorgente)
/Docs/PDR/Exports/*.pdf opzionale LFS o release artifact
```

Naming: RT_<Area>_<AssetName> e cartelle per dominio, non per autore.

11. Test

- 1 Esegui validator in Editor e commandlet CI.
- 2 Due scansioni del catalogo producono stesso manifest/hash.
- 3 Cook test verifica che tutti i Primary Assets richiesti siano inclusi.
- 4 Test rename/redirect esplicito e incompatibilità di versione.
- 5 Test JSON round-trip solo quando lo schema mod viene introdotto.
- 6 Test packaged server/client con manifest mismatch e messaggio diagnostico chiaro.

Fonti tecniche verificate: Epic Games, Asset Management in Unreal Engine, UE 5.8.; Epic Games, UPrimaryDataAsset C++ API, UE 5.8.; Epic Games, Using Gameplay Tags in Unreal Engine, UE 5.8.